جامعة الوادي — كلية العلوم الدقيقة — قسم الإعلام الآلي

ماستر 2 — الذكاء الاصطناعي وعلوم البيانات | 2026/2027

TP04
الحاويات ونشر بيئة تحليل البيانات باستخدام Docker

تقرير تطبيقي مُحدَّث بالنتائج النهائية والصور الأصلية

المقياس: **Big Data Analytics**

البيئة: **Windows 11 · Docker Desktop · WSL2**

مجموعة البيانات: **train.csv**

1 ـ المقدمة

يهدف هذا العمل التطبيقي إلى فهم تقنيات الحاويات باستخدام Docker، وبناء صورة لتطبيق Python يحلل مجموعة بيانات CSV من TP03، ونشر الصورة على Docker Hub، ثم تشغيل عدة حاويات واختبار شبكتها وربط المنافذ. كما يجري تشغيل التطبيق باستخدام Docker Compose.

2 ـ بيئة العمل والأدوات

الأداة	الإصدار أو الوصف
نظام التشغيل	Windows 11
Docker Desktop	4.96.3
Docker Engine	29.9.2
Docker Compose	5.6.1
Backend	WSL2
Python	3.11
المكتبة	Pandas
Dataset	data/train.csv
المحرر	Visual Studio Code

3 ـ المهمة الأولى — تثبيت Docker

تم تثبيت Docker Desktop وتشغيل Docker Engine باستخدام WSL2. استُخدمت الأوامر التالية للتأكد من توافر العميل والخادم:

```
docker version
docker info
```

أظهر فحص `docker version` معلومات العميل والخادم، وهو ما يوثّقه الشكل 1. كما تم إعداد حساب Docker Hub باسم alyatoumi.

4 ـ المهمة الثانية — بناء ونشر صورة Docker

4.1 تطبيق تحليل البيانات

يقرأ البرنامج analysis.py ملف train.csv، ويعرض عدد الصفوف والأعمدة، والقيم المفقودة والمكررة، وإجمالي المبيعات، وتحليل المبيعات حسب المنطقة والفئة والفئة الفرعية.

4.2 ملف Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY analysis.py .
COPY data/ ./data/
CMD ["python", "analysis.py"]
```

4.3 إنشاء الصورة وتجربتها

```
docker build -t tp04-analysis:1.0 .
docker run --rm tp04-analysis:1.0
```

4.4 النتائج

المؤشر	النتيجة
عدد الصفوف	9,800
عدد الأعمدة	18
القيم المفقودة	11
الصفوف المكررة	0
إجمالي المبيعات	2,261,536.78

المنطقة	إجمالي المبيعات
West	710,219.6845
East	669,518.7260
Central	492,646.9132
South	389,151.4590

الفئة	إجمالي المبيعات
Technology	827,455.8730
Furniture	728,658.5757
Office Supplies	705,422.3340

4.5 نشر الصورة إلى Docker Hub

```
docker tag tp04-analysis:1.0 alyatoumi/tp04-analysis:1.0
docker push alyatoumi/tp04-analysis:1.0
docker pull alyatoumi/tp04-analysis:1.0
```

تم رفع الصورة واسترجاعها بنجاح. يظهر المستودع وإصداره 1.0 في الشكل 2.

5 ـ المهمة الثالثة — الحاويات والشبكة

5.1 إنشاء شبكة تربط الحاويات

```
docker network create tp04-network
```

5.2 التشغيل الأول لتحليل البيانات

في البداية أُنشئت الحاويات الثلاث tp04-c1 وtp04-c2 وtp04-c3 من الصورة المحمّلة، وشُغلت باستخدام الأمر الافتراضي python analysis.py:

```
docker create --name tp04-c1 --network tp04-network alyatoumi/tp04-analysis:1.0
docker create --name tp04-c2 --network tp04-network alyatoumi/tp04-analysis:1.0
docker create --name tp04-c3 --network tp04-network alyatoumi/tp04-analysis:1.0
docker start tp04-c1 tp04-c2 tp04-c3
docker logs tp04-c1
docker logs tp04-c2
docker logs tp04-c3
```

أنجزت الحاويات الثلاث تحليل البيانات وانتهت بالحالة Exited (0)، وهي حالة طبيعية لتطبيق دفعي ينهي عمله بعد الحسابات.

5.3 إعادة إنشاء الحاويات الأصلية لاختبار HTTP

للتأكد من التشغيل المتزامن والاتصال الشبكي، حُذفت الحاويات الثلاث بعد حفظ نتائجها وأُعيد إنشاؤها **بالأسماء نفسها ومن الصورة نفسها** لتشغيل خوادم HTTP مستمرة، بدل إنشاء حاويات اختبار بأسماء مختلفة.

```
docker rm tp04-c1 tp04-c2 tp04-c3
docker run -d --name tp04-c1 --network tp04-network --entrypoint python alyatoumi/tp04-
analysis:1.0 -m http.server 8000
docker run -d --name tp04-c2 --network tp04-network --entrypoint python alyatoumi/tp04-
analysis:1.0 -m http.server 8000
docker run -d --name tp04-c3 --network tp04-network --entrypoint python alyatoumi/tp04-
analysis:1.0 -m http.server 8000
```

5.4 اختبار تشغيل الحاويات الثلاث

```
docker ps --filter "name=tp04-c"
```

أظهر Docker جميع الحاويات الثلاث في حالة Up كما يوضّح الشكل 3. وجود خادم HTTP مستمر أبقى الحاويات قيد التشغيل.

5.5 تنفيذ تحليل البيانات في الحاويات الثلاث نفسها

```
docker exec tp04-c1 python /app/analysis.py
docker exec tp04-c2 python /app/analysis.py
docker exec tp04-c3 python /app/analysis.py
```

نجح تشغيل تحليل البيانات داخل كل حاوية، وأظهرت الحاويات النتائج نفسها: 9,800 صف و18 عمودًا وإجمالي مبيعات 2,261,536.78. ظهرت الرسالة Analysis completed successfully! ثلاث مرات، وظلت الحاويات في حالة Up بعد انتهاء أوامر التحليل.

5.6 اختبار الاتصال بين الحاويات

```
docker exec tp04-c1 python -c "import urllib.request; r=urllib.request.urlopen('http://tp04-
c2:8000', timeout=5); print('HTTP Status:', r.status)"
```

كانت النتيجة HTTP Status: 200 كما يظهر في الشكل 4؛ وهذا يثبت نجاح الاتصال من tp04-c1 إلى tp04-c2 بواسطة اسم الحاوية داخل الشبكة.

5.7 اختبار Port Mapping

في اختبار مستقل، استُخدم الربط -p 8080:8000 مع حاوية مؤقتة لخادم HTTP، وأمكن فتح http://localhost:8080/ في متصفح Windows. هذا اختبار إضافي ولا يعني أن منافذ الحاويات الثلاث الحالية منشورة على الجهاز المضيف.

6 ـ المهمة الرابعة — Docker Compose

تم إنشاء الملف compose.yaml بالمواصفات المطلوبة:

```
services:
  data-analysis:
    build: .
    container_name: tp04-analysis
    volumes:
      - ./data:/app/data:ro
    restart: "no"
```

```
docker compose up --build
docker compose ps -a
docker compose down
```

نجح البناء والتشغيل، وعالج التطبيق ملف CSV ثم انتهت الحاوية برمز 0، كما توثّقه الصورة الخامسة. وقد أزال docker compose down حاوية Compose وشبكتها الافتراضية دون المساس بحاويات tp04-c1/c2/c3 المنشأة بصورة منفصلة.

السؤال 1: ما المشكلة التي يعالجها Docker Compose؟

يسهّل تعريف وإدارة الخدمات والشبكات ووحدات التخزين في ملف YAML واحد، بدل تكرار أوامر تشغيل عديدة يدويًا.

السؤال 2: مقارنة docker run وdocker compose up

ينشئ docker run حاوية فردية من خلال الخيارات المكتوبة في سطر الأوامر، بينما ينشئ docker compose up الخدمات المُعرّفة في ملف Compose ويديرها بطريقة قابلة لإعادة الاستخدام.

السؤال 3: كيف نضيف خدمة قاعدة بيانات؟

يمكن تعريف خدمة جديدة مثل PostgreSQL داخل compose.yaml، مع متغيرات بيئة ووحدة تخزين دائمة، ثم تهيئة برنامج Python للاتصال بها عبر اسم الخدمة في شبكة Compose.

7 ـ المهمة الخامسة — إجابات التقرير

ما Docker ولماذا يُستخدم في علوم البيانات؟

Docker منصة لتجميع التطبيقات واعتمادياتها في صور قابلة للنقل، وتُستخدم للحصول على بيئات تحليل قابلة لإعادة الإنتاج، ولتبسيط النقل والنشر، والحد من تعارض المكتبات والإصدارات.

الفرق بين Docker Image وDocker Container

الصورة Image قالب يحتوي على التطبيق ومتطلباته. أما الحاوية Container فهي نسخة تشغيلية من الصورة، ويمكن إنشاء عدة حاويات من صورة واحدة.

وظائف تعليمات Dockerfile

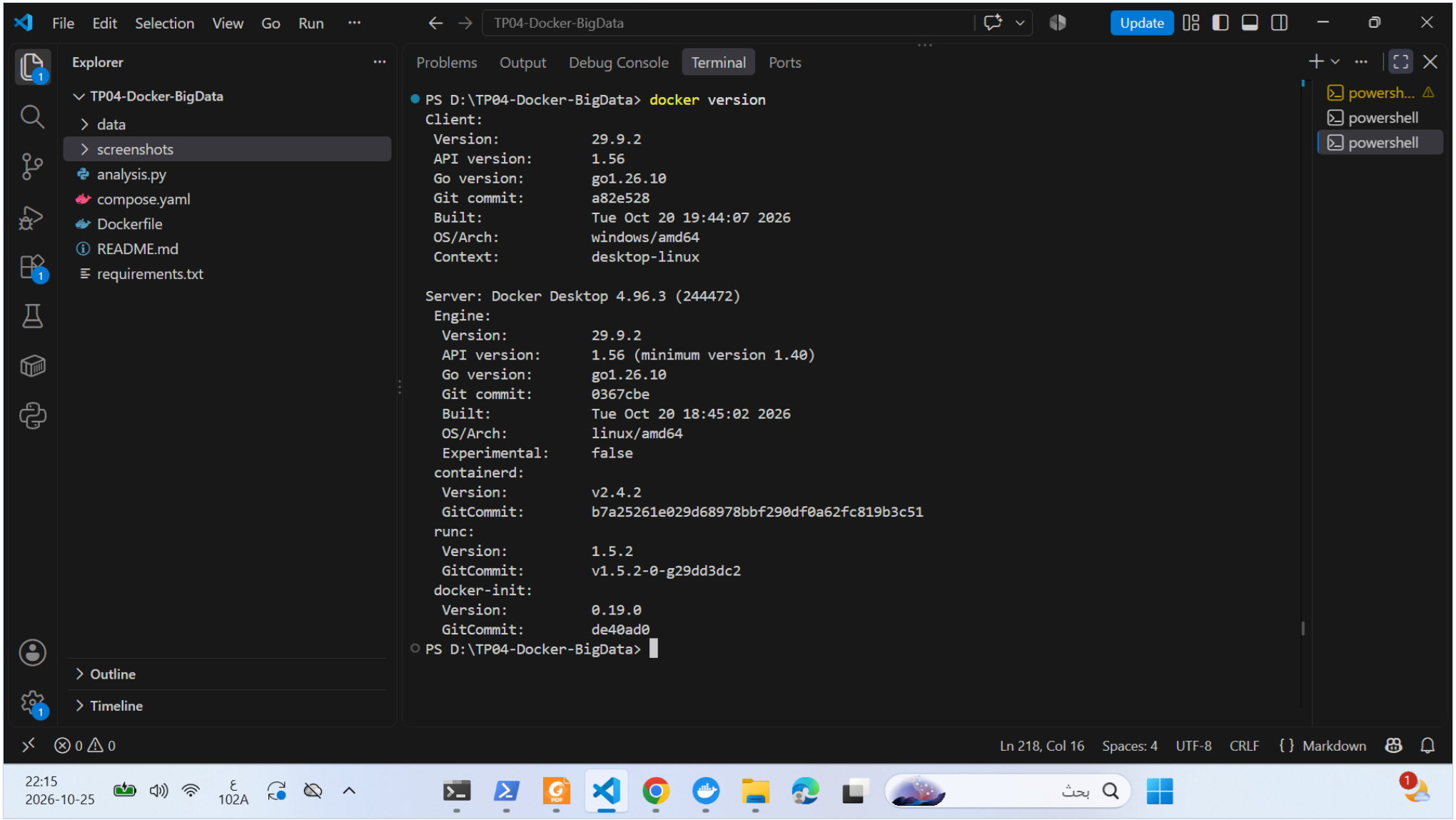
التعليمة	الوظيفة
FROM	تحديد الصورة الأساسية.
WORKDIR	تحديد مجلد العمل داخل الصورة.
COPY	نسخ ملفات المشروع إلى الصورة.
RUN	تنفيذ أوامر أثناء بناء الصورة، مثل تثبيت المكتبات.
CMD	تحديد أمر التشغيل الافتراضي للحاوية.

8 ـ الخاتمة

أثبتت التجربة نجاح إنشاء صورة Docker لتطبيق تحليلي بلغة Python ونشرها واسترجاعها من Docker Hub. واستُخدمت الحاويات الثلاث نفسها tp04-c1 وtp04-c2 وtp04-c3 للتحقق من التشغيل المتزامن ومن تنفيذ التحليل باستخدام docker exec. كما أثبت طلب HTTP بنتيجة 200 نجاح الاتصال الشبكي، وأظهر Docker Compose إمكانية تشغيل التطبيق اعتمادًا على ملف توصيف واحد.

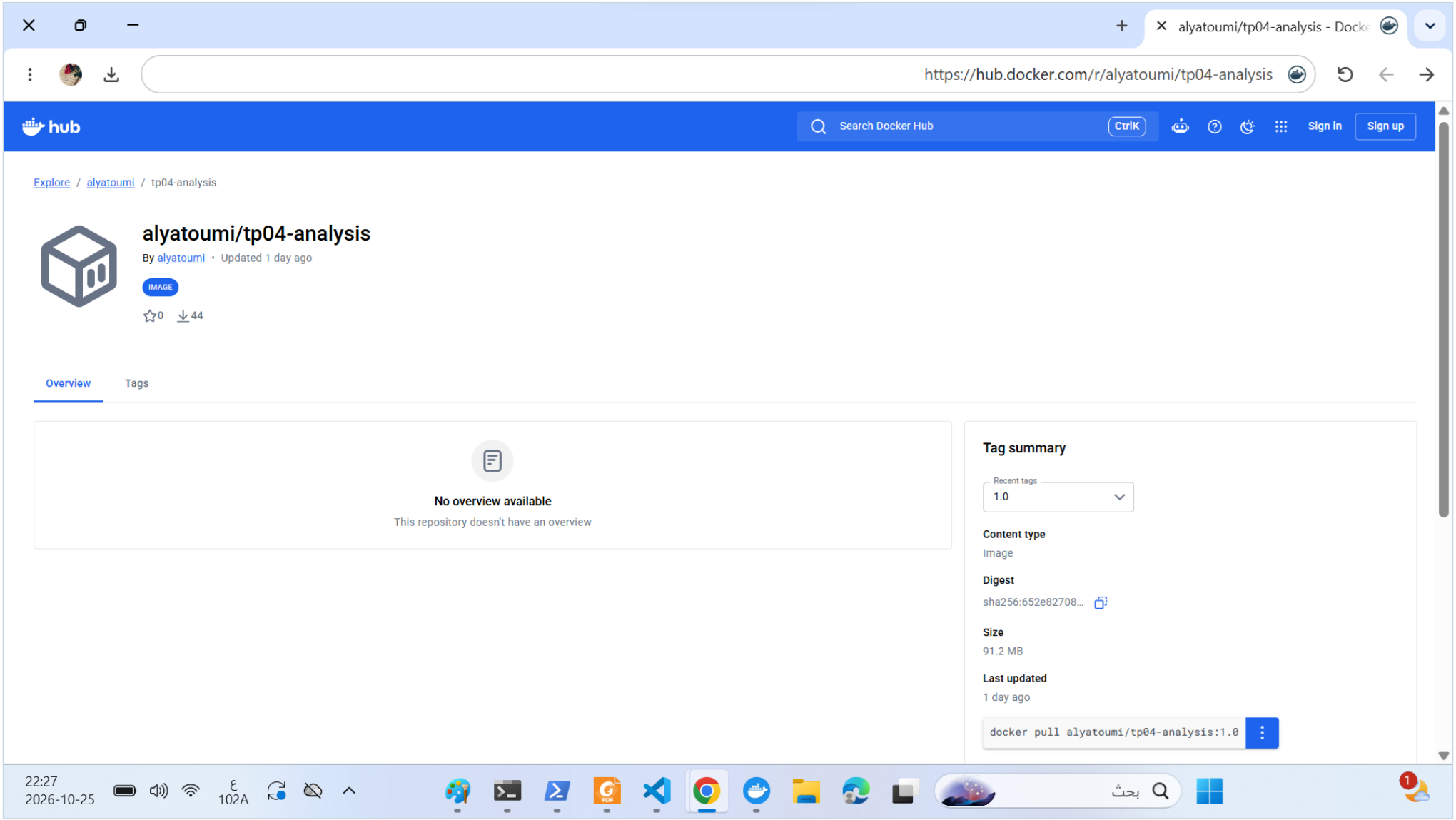
> **الصور التوثيقية:** أُدرجت في الصفحات التالية الصور الخمس التي قدّمها صاحب المشروع، دون تعديل محتواها أو استبدال النصوص فيها؛ عُرضت بالحفاظ على نسبة الأبعاد الأصلية.

الشكل 1: التحقق من تثبيت Docker — نتيجة الأمر docker version



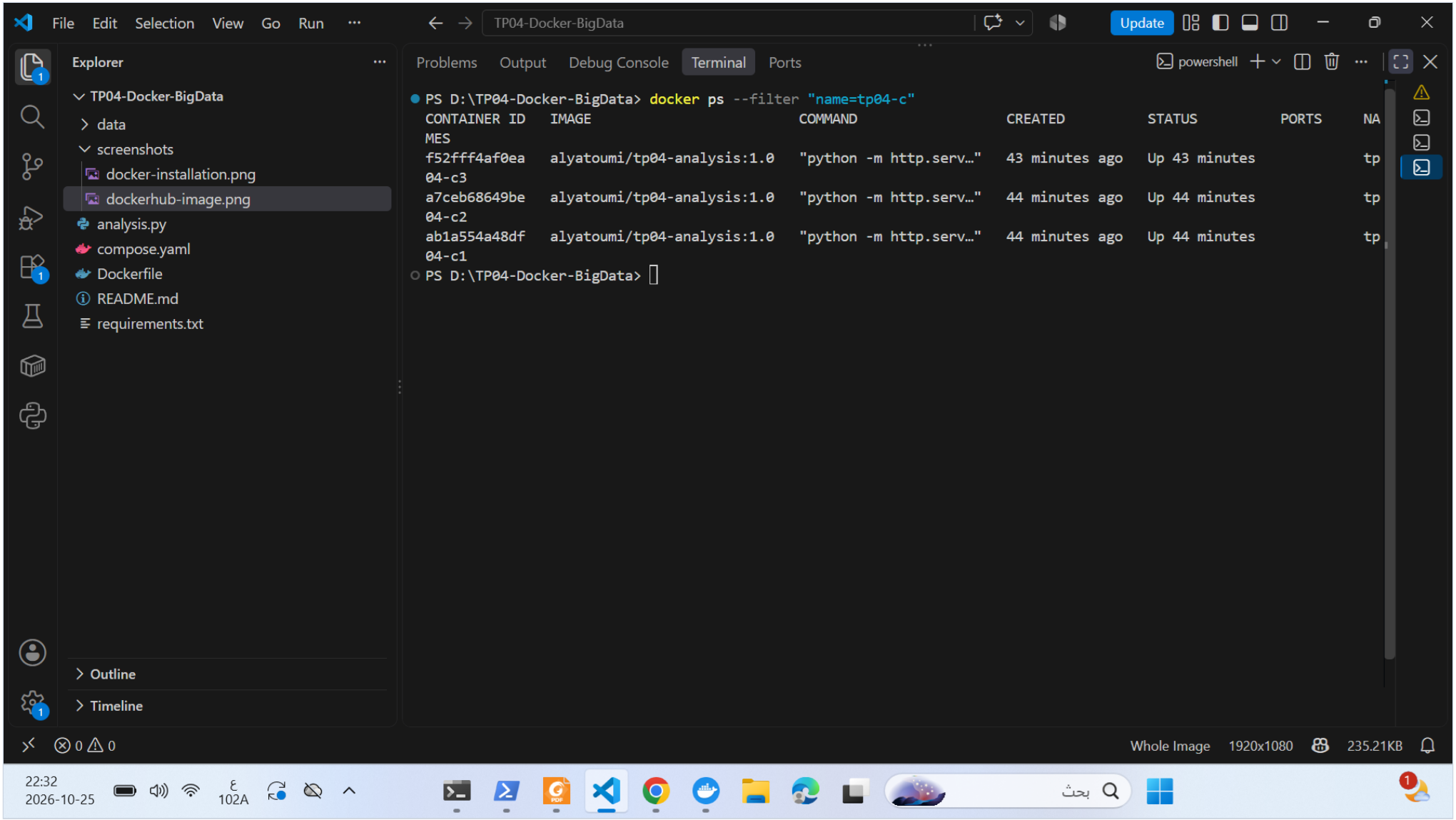
لقطة شاشة أصلية من تنفيذ TP04؛ تُعرض دون قص أو تمديد.

الشكل 2: صورة التطبيق المنشورة على Docker Hub بالإصدار 1.0



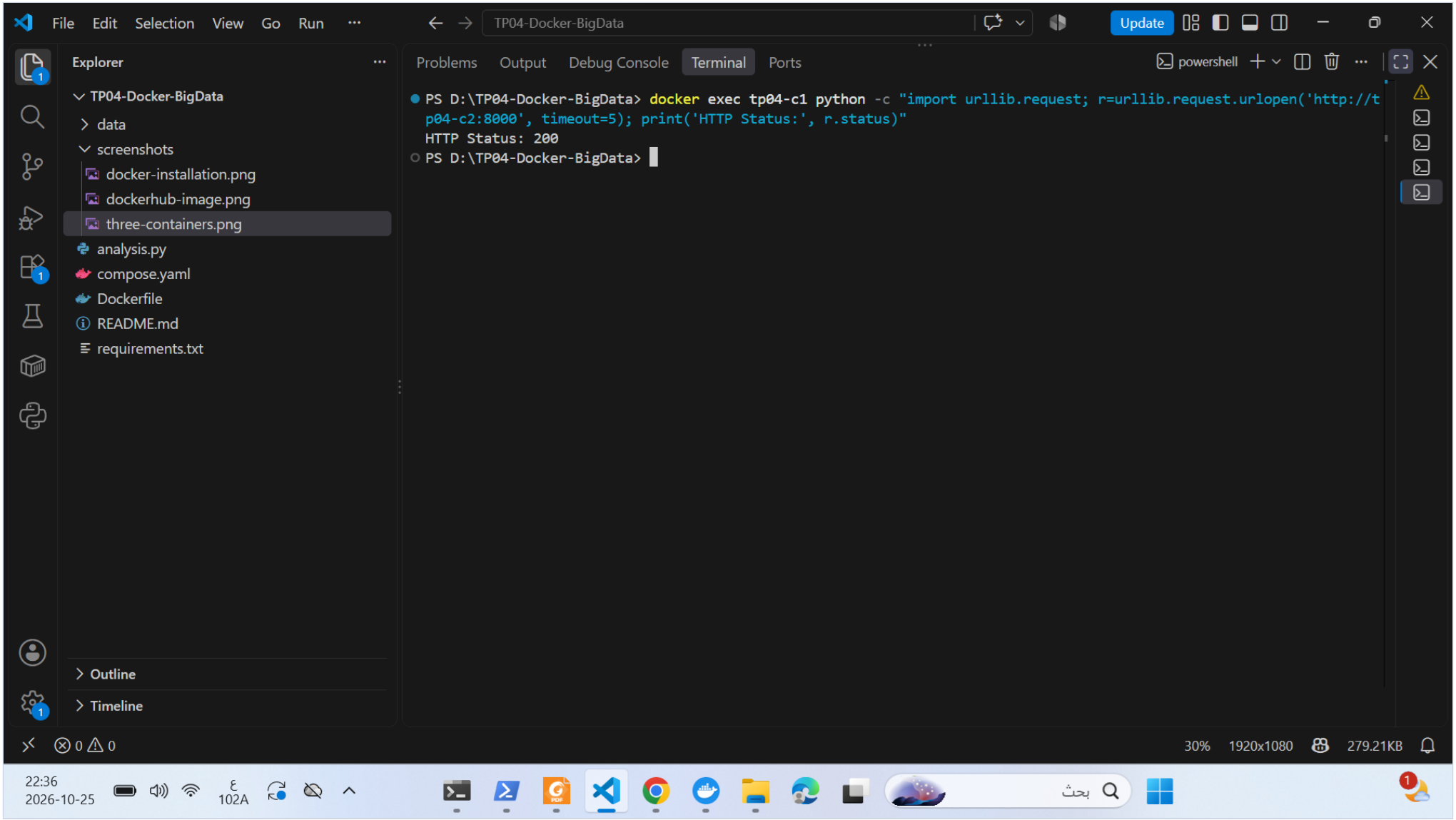
لقطة شاشة أصلية من تنفيذ TP04؛ تُعرض دون قص أو تمديد.

الشكل 3: تشغيل الحاويات الثلاث tp04-c1 وtp04-c2 وtp04-c3 بحالة Up



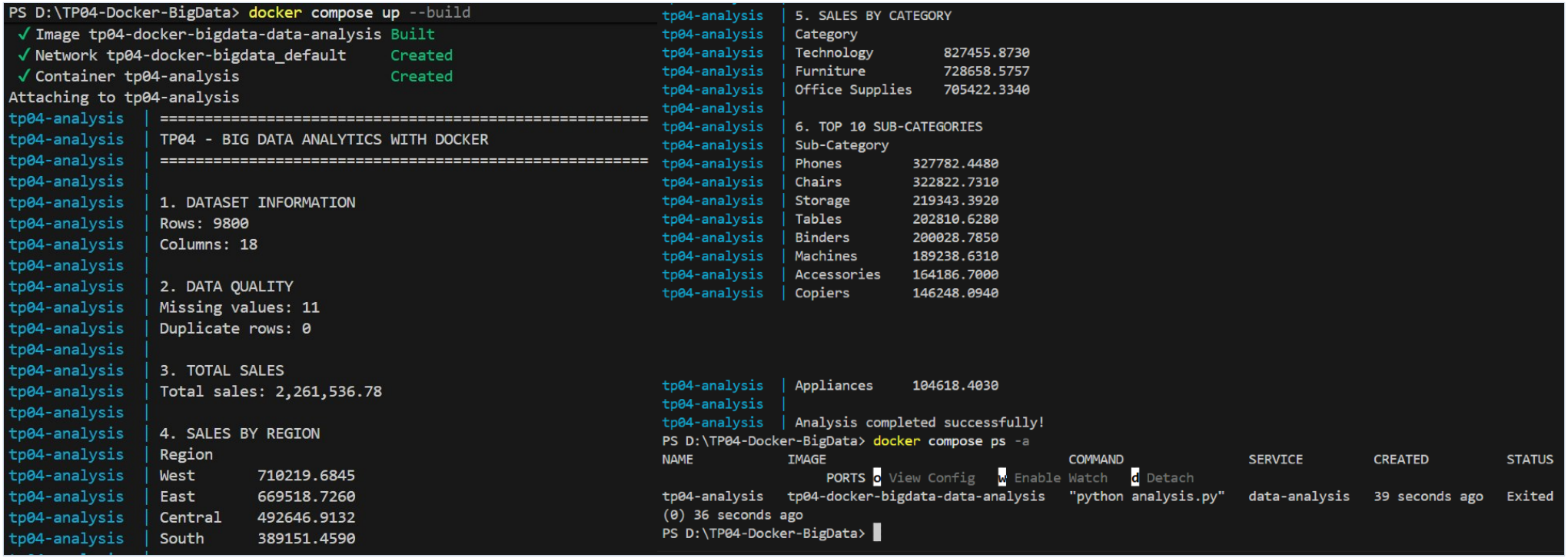
لقطة شاشة أصلية من تنفيذ TP04؛ تُعرض دون قص أو تمديد.

الشكل 4: نجاح الاتصال بين tp04-c1 و tp04-c2 — HTTP Status: 200



لقطة شاشة أصلية من تنفيذ TP04؛ تُعرض دون قص أو تمديد.

الشكل 5: تنفيذ Docker Compose ونجاح تحليل البيانات وانتهاء الحاوية برمز 0



لقطة شاشة أصلية من تنفيذ TP04؛ تُعرض دون قص أو تمديد.